

Computación Numérica y Visualización en Python

Guía Práctica de Uso: NumPy y Matplotlib

Desarrollo y Análisis Científico

18 de agosto de 2026

Resumen

En este documento se presenta una guía fundamental sobre dos de las librerías más importantes del ecosistema científico de Python: **NumPy** y **Matplotlib**. Se analizan los conceptos esenciales para la creación y manipulación de arreglos multidimensionales de alto rendimiento, operaciones vectorizadas y la generación de gráficos estáticos, personalización de figuras y visualización de datos experimentales y teóricos.

Índice

1. Introducción al Ecosistema Científico	1
2. Uso y Fundamentos de NumPy	2
2.1. Instalación e Importación	2
2.2. Creación de Arreglos (<code>ndarray</code>)	2
2.3. Operaciones Vectorizadas y Broadcasting	2
2.4. Estadística y Reducción de Dimensiones	3
3. Creación de Gráficas con Matplotlib	3
3.1. Instalación e Importación	3
3.2. Estructura de un Gráfico: Enfoque Orientado a Objetos	3
3.3. Tipos Principales de Gráficas	4
3.3.1. 1. Gráfico de Dispersión (<i>Scatter Plot</i>)	4
3.3.2. 2. Histogramas y Distribuciones	4
3.3.3. 3. Múltiples Subgráficos (<i>Subplots</i>)	4
4. Ejemplo Integrado: Simulación y Visualización	5
5. Recomendaciones y Buenas Prácticas	6
6. Conclusiones	6

1. Introducción al Ecosistema Científico

Python se ha consolidado como el lenguaje estándar para la ciencia de datos, machine learning e ingeniería gracias a su rico ecosistema de librerías especializadas. Entre ellas:

- **NumPy (*Numerical Python*)**: Es la base fundamental para el cálculo numérico, proveyendo estructuras de datos de arreglos multidimensionales eficientes (`ndarray`) y funciones matemáticas optimizadas en C y Fortran.
- **Matplotlib**: Es la librería de cabecera para la generación de gráficos 2D y 3D en Python, permitiendo un control total sobre el diseño, escalas, etiquetas y estilos de visualización.

2. Uso y Fundamentos de NumPy

2.1. Instalación e Importación

Para instalar la librería en su entorno local, se ejecuta:

```
1 pip install numpy
```

En código Python, por convención estándar internacional, se importa con el alias `np`:

```
1 import numpy as np
```

2.2. Creación de Arreglos (`ndarray`)

A diferencia de las listas nativas de Python, los arreglos de NumPy almacenan datos homogéneos en bloques contiguos de memoria, lo que permite operaciones extremadamente rápidas.

```
1 import numpy as np
2
3 # 1. Desde una lista tradicional
4 arr_1d = np.array([1, 2, 3, 4, 5])
5 arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
6
7 # 2. Arreglos con valores constantes o ceros/unos
8 ceros = np.zeros((3, 3))           # Matriz 3x3 de ceros
9 unos = np.ones((2, 4))            # Matriz 2x4 de unos
10 identidad = np.eye(3)             # Matriz identidad 3x3
11
12 # 3. Secuencias numéricas
13 rango = np.arange(0, 10, 2)        # [0, 2, 4, 6, 8] (inicio, fin, paso)
14 espaciado = np.linspace(0, 1, 5)  # 5 puntos equiespaciados entre 0 y 1
```

Listing 1: Formas básicas de crear arreglos en NumPy

2.3. Operaciones Vectorizadas y Broadcasting

NumPy permite aplicar operaciones elemento a elemento sin necesidad de utilizar bucles `for`:

```
1 x = np.array([1.0, 2.0, 3.0])
2
3 # Operaciones vectorizadas
4 y_sum = x + 10           # [11., 12., 13.]
5 y_cuad = x ** 2          # [1., 4., 9.]
6 y_sin = np.sin(x)        # Seno de cada elemento
```

2.4. Estadística y Reducción de Dimensiones

Tabla 1: Funciones estadísticas comunes en NumPy

Función	Descripción	Ejemplo
<code>np.mean(a)</code>	Calcula la media aritmética	<code>np.mean(arr)</code>
<code>np.std(a)</code>	Desviación estándar	<code>np.std(arr)</code>
<code>np.min(a) / np.max(a)</code>	Valor mínimo / máximo	<code>np.max(arr)</code>
<code>np.sum(a, axis=0)</code>	Suma a lo largo de un eje	<code>arr.sum(axis=0)</code>
<code>np.dot(a, b)</code>	Producto escalar o matricial	<code>a @ b</code>

3. Creación de Gráficas con Matplotlib

3.1. Instalación e Importación

```
1 pip install matplotlib
```

El módulo principal para crear gráficos rápidos y personalizados es `matplotlib.pyplot`:

```
1 import matplotlib.pyplot as plt
```

3.2. Estructura de un Gráfico: Enfoque Orientado a Objetos

Aunque existe una interfaz estilo MATLAB (`plt.plot(...)`) para pruebas rápidas, la mejor práctica en proyectos profesionales es utilizar el enfoque orientado a objetos con `Figure` y `Axes`.

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Creacion de la figura y el eje
5 fig, ax = plt.subplots(figsize=(8, 5), dpi=100)
6
7 # Generacion de datos
8 x = np.linspace(0, 2 * np.pi, 200)
9 y1 = np.sin(x)
10 y2 = np.cos(x)
11
12 # Graficar lineas
13 ax.plot(x, y1, label=r'$\sin(x)$', color='royalblue', linewidth=2)
14 ax.plot(x, y2, label=r'$\cos(x)$', color='crimson', linestyle='--',
15         linewidth=2)
16
17 # Personalizacion
18 ax.set_title("Ondas Trigonometricas", fontsize=14, fontweight='bold')
19 ax.set_xlabel("Eje X (Radianes)", fontsize=11)
20 ax.set_ylabel("Amplitud", fontsize=11)
```

```

20 ax.grid(True, linestyle=':', alpha=0.6)
21 ax.legend(loc='upper right')
22
23 # Mostrar o guardar
24 plt.tight_layout()
25 # plt.savefig("grafico_ondas.png", dpi=300) # Opcional: exportar a
    imagen
26 plt.show()

```

Listing 3: Plantilla base orientada a objetos en Matplotlib

3.3. Tipos Principales de Gráficas

3.3.1. 1. Gráfico de Dispersión (*Scatter Plot*)

Ideal para comparar relaciones entre dos variables continuas.

```

1 # Datos con dispersion aleatoria
2 np.random.seed(42)
3 x = np.random.rand(50)
4 y = 2 * x + np.random.normal(0, 0.2, 50)
5
6 fig, ax = plt.subplots()
7 ax.scatter(x, y, color='darkorange', alpha=0.7, edgecolors='black')
8 ax.set_title("Diagrama de Dispersion")
9 plt.show()

```

3.3.2. 2. Histogramas y Distribuciones

Útiles para observar la densidad de probabilidad o conteos por intervalos.

```

1 datos_normales = np.random.normal(loc=0, scale=1, size=1000)
2
3 fig, ax = plt.subplots()
4 ax.hist(datos_normales, bins=30, color='mediumseagreen', edgecolor='
    black', alpha=0.7)
5 ax.set_title("Histograma de Distribucion Normal")
6 ax.set_xlabel("Valores")
7 ax.set_ylabel("Frecuencia")
8 plt.show()

```

3.3.3. 3. Múltiples Subgráficos (*Subplots*)

Permite organizar múltiples vistas en una cuadrícula.

```

1 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
2
3 # Primer subgrafico
4 axes[0].plot(x, np.exp(x), color='purple')
5 axes[0].set_title("Crecimiento Exponencial")
6
7 # Segundo subgrafico
8 axes[1].plot(x, np.log(x + 1), color='teal')
9 axes[1].set_title("Crecimiento Logaritmico")
10
11 plt.tight_layout()
12 plt.show()

```

4. Ejemplo Integrado: Simulación y Visualización

El siguiente ejemplo muestra la integración de NumPy (para generar una señal modulada en amplitud con ruido) y Matplotlib (para graficar la señal limpia, el ruido y la señal combinada):

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 1. Definir parametros temporales
5 t = np.linspace(0, 1, 1000)      # 1 segundo, 1000 muestras
6
7 # 2. Generar componentes con NumPy
8 frecuencia = 5                    # 5 Hz
9 senal_pura = np.sin(2 * np.pi * frecuencia * t)
10 ruido = np.random.normal(0, 0.25, size=t.shape)
11 senal_contaminada = senal_pura + ruido
12
13 # 3. Graficar con Matplotlib
14 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 6), sharex=True)
15
16 # Panel superior: Senal limpia vs Senal ruidosa
17 ax1.plot(t, senal_contaminada, color='gray', alpha=0.6, label='Senal +
    Ruido')
18 ax1.plot(t, senal_pura, color='blue', linewidth=2, label='Senal Pura (5
    Hz)')
19 ax1.set_title("Análisis de Senal Senoidal con Ruido Gaussiano", fontsize
    =13)
20 ax1.set_ylabel("Amplitud")
21 ax1.grid(True, linestyle='--')
22 ax1.legend(loc='upper right')
23
24 # Panel inferior: Componente de Ruido
25 ax2.plot(t, ruido, color='crimson', linewidth=0.8, label='Ruido
    Gaussiano  $\mathcal{N}(0, \sigma^2)$ ')
26 ax2.set_xlabel("Tiempo (segundos)")
27 ax2.set_ylabel("Amplitud del Ruido")
28 ax2.grid(True, linestyle='--')
29 ax2.legend(loc='upper right')
30
31 plt.tight_layout()
32 plt.show()
```

Listing 4: Ejemplo integral: Señal con ruido

5. Recomendaciones y Buenas Prácticas

Buenas Prácticas

1. **Evitar bucles explícitos:** Siempre que sea posible, utilice operaciones vectorizadas de NumPy (`np.sin`, suma directa de arreglos, etc.) para aprovechar el paralelismo y la velocidad de C.
2. **Usar la API Orientada a Objetos:** Prefiera `fig`, `ax = plt.subplots()` sobre llamadas directas de `plt.plot()` para proyectos medianos o grandes.
3. **Control de Memoria y Cierre de Figuras:** Si genera múltiples gráficos en bucle o reportes automáticos, utilice `plt.close(fig)` para liberar memoria RAM.
4. **Configuración de Estilos:** Puede aplicar temas predefinidos con `plt.style.use('seaborn-v0_8-whitegrid')` o `plt.style.use('ggplot')`.

6. Conclusiones

La combinación de **NumPy** para el cómputo algebraico y matricial junto con **Matplotlib** para la representación visual constituye una de las herramientas más potentes y versátiles para el análisis numérico, la investigación científica y el desarrollo de modelos matemáticos.

El Movimiento Rectilíneo en la Física

Fundamentos teóricos, modelos matemáticos, metodología de resolución paso a paso y aplicaciones clave en ingeniería y ciencias.

Documento: Guía Técnico-Didáctica **Disciplina:** Mecánica Clásica / Cinemática

Estructura: Fundamentos, Método Paso a Paso, Ejemplos y Aplicaciones Reales

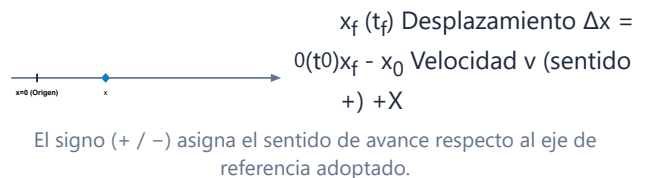
1. ¿Qué es el Movimiento Rectilíneo? Conceptos Fundamentales

En la física clásica, el **Movimiento Rectilíneo** es el tipo más elemental de movimiento mecánico. Se define como aquel en el cual un cuerpo (modelado como partícula puntual) se desplaza a lo largo de una **trayectoria estrictamente unidimensional** (una línea recta geométrica). Al ser la dirección constante o colineal a lo largo del tiempo, las magnitudes vectoriales se manipulan algebraicamente mediante números reales con signo referidos a un eje de coordenadas.

Magnitudes Cinemáticas Fundamentales

- Posición (x):** Coordenada del móvil con respecto a un origen fijo $x = 0$ en un instante t (unidad SI: metros, m).
- Desplazamiento (Δx):** Cambio neto de posición: $\Delta x = x_f - x_0$. Es un vector con signo (+ o -).
- Distancia (d):** Longitud escalar total recorrida sobre la recta (siempre positiva, $d \geq |\Delta x|$).
- Velocidad instantánea (v):** Tasa de variación de la posición respecto al tiempo: $v = dx/dt$ (unidad SI: m/s).
- Aceleración instantánea (a):** Tasa de cambio de la velocidad en el tiempo: $a = dv/dt = d^2x/dt^2$ (unidad SI: m/s²).

Esquema Vectorial Unidimensional



2. Clasificación y Modelos Matemáticos

A. Movimiento Rectilíneo Uniforme (MRU)

$a = 0$

Velocidad estrictamente constante. Aceleración nula. El móvil recorre distancias idénticas en lapsos de tiempo iguales.

$$x(t) = x_0 + v \cdot t$$

- Gráfica x vs t :** Línea recta cuya pendiente representa v .
- Gráfica v vs t :** Recta horizontal; el área bajo la curva es Δx .

B. Movimiento Rectilíneo Unif. Variado (MRUA)

$a = \text{const} \neq 0$

Aceleración constante. La velocidad cambia linealmente en el tiempo y la posición varía de forma cuadrática.

$$x(t) = x_0 + v_0 \cdot t + \frac{1}{2} a \cdot t^2$$

- Gráfica v vs t :** Recta inclinada cuya pendiente es a .
- Gráfica x vs t :** Curva parabólica continua.

Ecuaciones Cinemáticas Canónicas del MRUA

1. VELOCIDAD - TIEMPO

$$v(t) = v_0 + a \cdot t$$

2. POSICIÓN - TIEMPO

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2$$

3. INDEPENDIENTE DEL TIEMPO (TORRICELLI)

$$v_f^2 = v_0^2 + 2 a \Delta x$$

4. DESPLAZAMIENTO POR VELOCIDAD MEDIA

$$\Delta x = \frac{1}{2}(v_0 + v_f) \cdot t$$

Caída Libre y Tiro Vertical (Gravedad Terrestre): Corresponde a un MRUA unidimensional a lo largo del eje vertical y donde la aceleración es debida exclusivamente a la gravedad: $a = -g$ (con $g \approx 9.81 \text{ m/s}^2$). Se aplican idénticas ecuaciones sustituyendo $x \rightarrow y$ y $a \rightarrow -g$.

3. ¿CÓMO se usa el Movimiento Rectilíneo? Metodología Paso a Paso

Para resolver cualquier problema cinemático o modelar sistemas en ingeniería sin errores conceptuales, se aplica el siguiente algoritmo de 5 pasos:

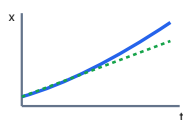
- Paso 1: Definir el Sistema de Referencia (SR) y Convención de Signos**
Fija el origen espacial $x = 0$ y el instante temporal inicial $t = 0$. Define la orientación positiva del eje (generalmente hacia la derecha o hacia arriba). Recuerda: posición, velocidad y aceleración son magnitudes con signo.
- Paso 2: Inventario de Parámetros y Extracción de Datos**
Organiza las 5 variables cinemáticas fundamentales: **Posición inicial** (x_0), **Posición final** (x_f o Δx), **Velocidad inicial** (v_0), **Velocidad final** (v_f), **Aceleración** (a) y **Tiempo** (t). Separa claramente los datos conocidos de la incógnita buscada.
- Paso 3: Selección Estratégica de la Ecuación Cinemática**
Examina cuál de las 5 variables *no aparece en los datos ni se solicita calcular*. Selecciona la ecuación cinemática que excluye exactamente esa variable innecesaria (ver matriz de decisión abajo).
- Paso 4: Estandarización de Unidades y Despeje Algebraico**
Convierte todas las magnitudes al **Sistema Internacional (SI)**: distancias en metros (m), tiempos en segundos (s) y velocidades en m/s ($1 \text{ km/h} = 1/3.6 \text{ m/s}$). Despeja la variable incógnita de forma simbólica antes de sustituir números.
- Paso 5: Análisis Dimensional y Validación Crítica del Resultado**
Verifica que la unidad obtenida sea coherente y que los signos concuerden con la física del problema (frenado $\Rightarrow$ aceleración de signo opuesto a la velocidad; altura máxima $\Rightarrow v = 0$).

Matriz de Decisión para Selección de Ecuaciones (MRUA)

Ecuación	Variable Ausente	Criterio de Aplicación Óptimo
$v_f = v_0 + a \cdot t$	Desplazamiento (Δx)	Ideal cuando no se conoce ni se requiere calcular la distancia recorrida.
$\Delta x = v_0 \cdot t + \frac{1}{2} a \cdot t^2$	Velocidad Final (v_f)	Cuando se calcula la distancia conociendo aceleración y tiempo de trayecto.
$v_f^2 = v_0^2 + 2a\Delta x$	Tiempo (t)	Fundamental en distancias de frenado o aceleración donde el tiempo es desconocido.
$\Delta x = \frac{1}{2}(v_0 + v_f) \cdot t$	Aceleración (a)	Cuando se conocen ambas velocidades y el lapso de tiempo, sin requerir la aceleración.

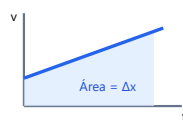
Interpretación Gráfica del Movimiento Rectilíneo

Posición vs. Tiempo ($x - t$)



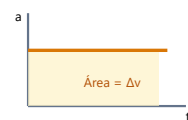
Pendiente = Velocidad (v).
Recta = MRU, Parábola = MRUA.

Velocidad vs. Tiempo ($v - t$)



Pendiente = Aceleración (a).
Área bajo la curva = Δx .

Aceleración vs. Tiempo ($a - t$)



Área bajo la curva = Δv .
Recta horizontal = MRUA.

4. Ejemplos Prácticos Resueltos Paso a Paso

EJEMPLO 1: Problema de Encuentro / Alcance entre dos Móviles

MRU

Enunciado: Dos vehículos parten simultáneamente al encuentro desde dos ciudades separadas por $d = 600 \text{ m}$ en línea recta. El Auto A se mueve a $v_A = 20 \text{ m/s}$ constante hacia la derecha. El Auto B avanza hacia la izquierda a $v_B = 10 \text{ m/s}$ constante. ¿En qué tiempo y posición se cruzan?

Paso 1 (Sistema de Referencia): Origen en la posición del Auto A: $x_{0A} = 0 \text{ m}$, $v_A = +20 \text{ m/s}$. Posición del Auto B: $x_{0B} = 600 \text{ m}$, $v_B = -10 \text{ m/s}$ (signo negativo porque se mueve hacia el origen).

Paso 2 y 3 (Ecuaciones Horarias MRU): $x_A(t) = 0 + 20t$ | $x_B(t) = 600 - 10t$.

Paso 4 (Condición de Encuentro): Se encuentran cuando $x_A(t_e) = x_B(t_e)$:
 $20 t_e = 600 - 10 t_e \Rightarrow 30 t_e = 600 \Rightarrow t_e = 20 \text{ s}$.

Paso 5 (Posición de Encuentro): Sustituir en cualquiera de las ecuaciones: $x_e = x_A(20) = 20(20) = 400 \text{ m}$.

• Tiempo de encuentro: $t_e = 20$ segundos

• Posición respecto al origen A: $x_e = 400$ metros

EJEMPLO 2: Seguridad Vial - Distancia Total de Detención de Emergencia

MRU + MRUA

Enunciado: Un automóvil circula a 108 km/h . El conductor percibe un obstáculo y tarda $t_r = 0.75 \text{ s}$ en pisar el freno. Una vez activados los frenos, el coche desacelera a razón constante de $a = -6.0 \text{ m/s}^2$ hasta el reposo ($v_f = 0$). Calcula la distancia total recorrida.

Paso 1 (Conversión de Unidades SI): $v_0 = 108 \text{ km/h} = (108 / 3.6) \text{ m/s} = 30.0 \text{ m/s}$.

Paso 2 (Fase 1: Reacción - MRU): Mientras reacciona, el auto mantiene velocidad constante:
 $d_{\text{reacc}} = v_0 \cdot t_r = 30.0 \text{ m/s} \times 0.75 \text{ s} = 22.5 \text{ m}$.

Paso 3 (Fase 2: Frenado - MRUA sin tiempo): Aplicando la ecuación de Torricelli:
 $v_f^2 = v_0^2 + 2 a d_{\text{fren}} \Rightarrow 0^2 = (30.0)^2 + 2(-6.0) d_{\text{fren}} \Rightarrow 12.0 d_{\text{fren}} = 900 \Rightarrow d_{\text{fren}} = 75.0 \text{ m}$.

Paso 4 (Distancia Total de Parada): $d_{\text{total}} = d_{\text{reacc}} + d_{\text{fren}} = 22.5 \text{ m} + 75.0 \text{ m} = 97.5 \text{ m}$.

• Distancia de reacción: 22.5 m | Distancia de frenado: 75.0 m

• Distancia Total de Parada: 97.5 metros

EJEMPLO 3: Balística Vertical - Altura Máxima y Tiempo de Vuelo

CAÍDA LIBRE / MRUA

Enunciado: Se dispara verticalmente hacia arriba un proyectil con velocidad inicial $v_0 = 29.43 \text{ m/s}$ desde el suelo ($y_0 = 0$). Tomando $g = 9.81 \text{ m/s}^2$ y despreciando la resistencia del aire, calcula la altura máxima alcanzada y el tiempo total de vuelo.

Paso 1 (Condición de Altura Máxima): En la cúspide de la trayectoria, la velocidad instantánea se anula: $v(t_{\text{sub}}) = 0 \text{ m/s}$.

Paso 2 (Tiempo de Subida): $v(t) = v_0 - g t \Rightarrow 0 = 29.43 - 9.81 t_{\text{sub}} \Rightarrow t_{\text{sub}} = 29.43 / 9.81 = 3.0 \text{ s}$.

Paso 3 (Cálculo de Altura Máxima): $y_{\text{max}} = v_0 t_{\text{sub}} - \frac{1}{2} g t_{\text{sub}}^2 = (29.43)(3.0) - 0.5(9.81)(3.0)^2 = 88.29 - 44.145 = 44.145 \text{ m}$.

Alternativa por Torricelli: $v_f^2 = v_0^2 - 2g y_{\text{max}} \Rightarrow 0 = (29.43)^2 - 2(9.81) y_{\text{max}} \Rightarrow y_{\text{max}} = 44.145 \text{ m}$.

Paso 4 (Tiempo Total de Vuelo): Por simetría en caída libre: $t_{\text{vuelo}} = 2 \cdot t_{\text{sub}} = 2 \times 3.0 \text{ s} = 6.0 \text{ s}$.

• Altura Máxima: $y_{\text{max}} = 44.15$ metros

• Tiempo Total de Vuelo: $t_{\text{vuelo}} = 6.0$ segundos

5. ¿PARA QUÉ se usa el Movimiento Rectilíneo en la Física y en el Mundo Real?

El estudio del movimiento rectilíneo no es una simple abstracción académica, sino la **herramienta matemática e ingenieril fundamental** para el diseño, control y análisis en múltiples ramas de la ciencia y tecnología:



1. Industria Aeroespacial y Cohetería

En el despegue vertical de cohetes espaciales (Falcon 9, SLS), la fase de ascenso lineal inicial se modela como MRUA para estimar las cargas estructurales g y optimizar la tasa de consumo de propelente.



2. Seguridad Vial e Infraestructura

Determinación de distancias de seguridad en autopistas, dimensionamiento de pistas de despegue y aterrizaje de aviones, diseño de zonas de absorción de impactos y calibración de sistemas antibloqueo ABS.



3. Robótica, CNC y Mecatrónica

Fresadoras CNC, impresoras 3D y brazos industriales ejecutan trayectorias complejas desacoplándolas en movimientos rectilíneos independientes controlados sobre los ejes ortogonales X, Y, Z .



4. Aceleradores de Partículas y Medicina Nuclear

Los aceleradores lineales (LINAC) emplean campos eléctricos en línea recta para acelerar partículas subatómicas para investigación nuclear y generación de radioterapia médica de precisión contra tumores.



5. Descomposición de Movimientos 2D y 3D (Galileo)

Movimientos complejos como el tiro parabólico se modelan como dos movimientos rectilíneos simultáneos e independientes: un **MRU en el eje X** y un **MRUA en el eje Y** (Principio de Superposición).



6. Biomecánica y Alto Rendimiento Deportivo

Análisis cinemático de atletas de velocidad en carreras de 100m lisos para estudiar la fase de aceleración inicial desde tacos, la velocidad punta y el decaimiento de rendimiento por fatiga.

6. Conexión Fundamental con la Dinámica y la Energía

Relación con las Leyes de Newton (Dinámica)

El movimiento rectilíneo constituye el vínculo directo entre la cinemática descriptiva y las causas de las fuerzas:

$$\Sigma F_x = m \cdot a_x$$

Fuerza neta nula ($\Sigma F=0$) $\Rightarrow$ MRU (1ª Ley / Inercia). Fuerza neta constante $\Rightarrow$ MRUA (2ª Ley de Newton).

Teorema del Trabajo y la Energía Cinética

Al multiplicar la ecuación de Torricelli ($v_f^2 - v_0^2 = 2a\Delta x$) por $\frac{1}{2}m$ se obtiene directamente la conservación de energía:

$$W_{\text{neto}} = F \cdot \Delta x = \frac{1}{2} m v_f^2 - \frac{1}{2} m v_0^2 = \Delta E_k$$

El trabajo mecánico neto aplicado sobre la recta se transforma íntegramente en variación de energía cinética.

7. Tabla Maestra de Fórmulas y Criterios de Signos

Tipo de Movimiento	Condición Aceleración	Ecuación de Velocidad	Ecuación de Posición	Ecuación Sin Tiempo
MRU	$a = 0$	$v = \text{constante}$	$x(t) = x_0 + v \cdot t$	$\Delta x = v \cdot t$
MRUA / MRUV	$a = \text{const} \neq 0$	$v(t) = v_0 + a \cdot t$	$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2$	$v_f^2 = v_0^2 + 2a\Delta x$
Caída Libre / Tiro Vert.	$a = -g (\approx -9.81 \text{ m/s}^2)$	$v(t) = v_0 - g \cdot t$	$y(t) = y_0 + v_0 t - \frac{1}{2} g t^2$	$v_f^2 = v_0^2 - 2g\Delta y$

MANEJO Y CONVERSIÓN DE UNIDADES QUÍMICAS

Fundamentos Teóricos, Método de Factores de Conversión y Aplicaciones Prácticas Paso a Paso

Incluye Compendio Exhaustivo de Unidades de Medida:
Matemáticas • Química • Física • Biología

Resumen de la Guía

Este documento proporciona una guía rigurosa, clara y pedagógica para comprender y operar con las unidades fundamentales y derivadas de la química. Se detalla el análisis dimensional mediante factores de conversión, el manejo de la cantidad de sustancia (*mol*), masa molar, volumen molar, número de entidades y las distintas formas de expresar la concentración de soluciones (Molaridad, Molalidad, Normalidad, Fracción Molar, ppm y porcentajes). Finalmente, se presenta un compendio comparativo de unidades para las ciencias exactas y naturales.

Documento Técnico y Didáctico de Referencia

Compilado en L^AT_EX • 18 de agosto de 2026

Índice

1. Introducción al Manejo de Unidades en Química	2
2. Conceptos Clave de la Cantidad de Sustancia	2
2.1. El Mol y el Número de Avogadro (N_A)	2
2.2. Masa Molar (M)	2
2.3. Volumen Molar de Gases Ideales (V_m)	2
3. El Método del Factor de Conversión Paso a Paso	3
3.1. Algoritmo Sistemático de 5 Pasos	3
4. Ejemplos Prácticos Resueltos y Explicados	3
5. Unidades de Concentración en Química	5
6. Tablas Comparativas de Unidades por Disciplina Científica	7
6.1. 1. Tabla de Unidades para Matemáticas e Informática Teórica	7
6.2. 2. Tabla de Unidades para Química y Fisicoquímica	7
6.3. 3. Tabla de Unidades para Física	8
6.4. 4. Tabla de Unidades para Biología y Ciencias de la Vida	8
7. Prefijos del Sistema Internacional de Unidades	11
8. Consejos Prácticos para Resolución de Problemas	11

1. Introducción al Manejo de Unidades en Química

En química cuantitativa, toda medición experimental carece de significado físico si no se acompaña de su correspondiente **unidad de medida**. A diferencia del álgebra pura, donde las variables representan números abstractos, en las ciencias empíricas cada magnitud describe una propiedad física o química del universo observable (masa, volumen, concentración, temperatura, presión o energía).

Regla de Oro del Análisis Dimensional

Toda cantidad medible se compone de un valor numérico y una dimensión física:

$$\text{Magnitud} = \text{Valor Numérico} \times [\text{Unidad}]$$

- **Suma y Resta:** Sólo se pueden sumar o restar magnitudes que compartan *idéntica dimensión y unidad*.
- **Multiplicación y División:** Las unidades se operan algebraicamente aplicando las reglas de los exponentes (cancelación, potencias, razones compuestas).

2. Conceptos Clave de la Cantidad de Sustancia

El enlace fundamental entre la escala microscópica (átomos, iones y moléculas) y la escala macroscópica (gramos, litros) en el laboratorio descansa sobre tres conceptos esenciales:

2.1. El Mol y el Número de Avogadro (N_A)

El **mol** es la unidad base del Sistema Internacional (SI) para la magnitud *cantidad de sustancia* (n).

- **Definición oficial (IUPAC / SI):** Un mol contiene exactamente $6.022\,140\,76 \times 10^{23}$ entidades elementales especificadas.
- **Factor de conversión fundamental:**

$$1 \text{ mol} = 6.022 \times 10^{23} \text{ entidades (átomos, moléculas, iones o electrones)}$$

2.2. Masa Molar (M)

La masa molar es la masa por unidad de cantidad de sustancia de una especie química dada. Se expresa convencionalmente en gramos por mol (g/mol).

$$n = \frac{m}{M} \quad \Longleftrightarrow \quad m = n \cdot M$$

donde n es la cantidad de sustancia en moles (mol), m es la masa en gramos (g) y M es la masa molar (g/mol).

2.3. Volumen Molar de Gases Ideales (V_m)

A condiciones determinadas de presión y temperatura, un mol de cualquier gas con comportamiento ideal ocupa un volumen idéntico:

- **Condiciones Normales Clásicas (CN: $T = 0^\circ\text{C} = 273.15 \text{ K}$, $P = 1 \text{ atm}$):**

$$V_m = 22.414 \text{ L/mol} \approx 22.4 \text{ L/mol}$$

- **Condiciones Estándar IUPAC (STP: $T = 0^\circ\text{C} = 273.15 \text{ K}$, $P = 1 \text{ bar} = 100 \text{ kPa}$):**

$$V_m = 22.711 \text{ L/mol}$$

▪ **Ecuación de Estado de Gases Ideales:**

$$P \cdot V = n \cdot R \cdot T \implies V = \frac{n \cdot R \cdot T}{P}$$

$$\text{con } R = 0.08206 \frac{\text{atm} \cdot \text{L}}{\text{mol} \cdot \text{K}} = 8.314 \frac{\text{J}}{\text{mol} \cdot \text{K}}.$$

3. El Método del Factor de Conversión Paso a Paso

El método del factor de conversión (o análisis dimensional unitario) es el procedimiento más robusto para realizar transformaciones de unidades sin recurrir a reglas de tres ambiguas o propensas a errores de inversión.

Definición de Factor Unitario de Conversión

Un factor de conversión es una relación fraccionaria cuyo valor intrínseco es igual a 1, originada a partir de una equivalencia demostrada:

$$\text{Cantidad Inicial [Unidad A]} \times \left(\frac{\text{Unidad B Deseada}}{\text{Unidad A Inicial}} \right) = \text{Cantidad Final [Unidad B]}$$

Al colocar la *Unidad A* en el denominador, ésta se simplifica algebraicamente con la unidad del término de partida.

3.1. Algoritmo Sistemático de 5 Pasos

Paso 1: Definir los datos y la meta: Escribir el dato de partida (número, unidad y compuesto químico) y especificar con claridad la unidad final solicitada.

Paso 2: Establecer la ruta de conversión: Determinar los puentes de enlace. En química, el **mol** constituye el nodo central de conversión:

$$\text{Masa (g)} \longleftrightarrow \text{Cantidad (mol)} \longleftrightarrow \begin{cases} \text{Nº de Entidades (átomos/moléculas)} \\ \text{Volumen de Gas a CN (L)} \\ \text{Volumen de Solución (L, vía Molaridad)} \end{cases}$$

Paso 3: Construir los factores de conversión unitarios: Disponer cada factor de forma que la unidad a eliminar se ubique en el lado opuesto (numerador frente a denominador).

Paso 4: Simplificar algebraicamente las dimensiones: Cancelar las unidades intermedias idénticas y comprobar que la única unidad remanente sea la deseada.

Paso 5: Operar numéricamente y evaluar cifras significativas: Multiplicar numeradores, dividir sucesivamente por denominadores y redondear según la precisión de los datos.

4. Ejemplos Prácticos Resueltos y Explicados

Ejemplo 1: Conversión de Masa a Número de Moléculas y Átomos

Enunciado: Una muestra contiene 90.0 g de glucosa pura ($\text{C}_6\text{H}_{12}\text{O}_6$, masa molar $M = 180.16 \text{ g/mol}$).

- ¿Cuántos moles de glucosa contiene?
- ¿Cuántas moléculas de glucosa contiene?
- ¿Cuántos átomos de carbono (C) están presentes en total?

Resolución Paso a Paso:

■ **Parte a) Cálculo de Moles:**

$$n = 90.0 \text{ g glucosa} \times \left(\frac{1 \text{ mol glucosa}}{180.16 \text{ g glucosa}} \right) = \mathbf{0.4996 \text{ mol glucosa}}$$

■ **Parte b) Cálculo de Moléculas (vía Número de Avogadro):**

$$N_{\text{moléc}} = 0.4996 \text{ mol} \times \left(\frac{6.022 \times 10^{23} \text{ moléculas}}{1 \text{ mol}} \right) = \mathbf{3.008 \times 10^{23} \text{ moléculas}}$$

■ **Parte c) Cálculo de Átomos de Carbono:** Dado que 1 molécula de glucosa = 6 átomos de C:

$$N_{\text{átomos C}} = 3.008 \times 10^{23} \text{ moléc.} \times \left(\frac{6 \text{ átomos C}}{1 \text{ moléc.}} \right) = \mathbf{1.805 \times 10^{24} \text{ átomos de C}}$$

Ejemplo 2: Transformación de Porcentaje en Masa (% m/m) y Densidad a Molaridad

Enunciado: Un frasco de reactivo de ácido clorhídrico concentrado (HCl, $M = 36.46 \text{ g/mol}$) indica en su etiqueta 37.0 % m/m y una densidad de $\rho = 1.19 \text{ g/mL}$. Determinar la **Molaridad** (M) de la solución.

Resolución Paso a Paso:

■ **Interpretación de los datos:**

- $37.0 \% \text{ m/m} \Rightarrow \frac{37.0 \text{ g HCl}}{100 \text{ g solución}}$.
- $\rho = 1.19 \text{ g/mL} \Rightarrow \frac{1.19 \text{ g solución}}{1 \text{ mL solución}} = \frac{1190 \text{ g solución}}{1 \text{ L solución}}$.
- Molaridad buscada: $M = \frac{\text{mol HCl}}{\text{L solución}}$.

■ **Cadena dimensional continua:**

$$M = \left(\frac{37.0 \text{ g HCl}}{100 \text{ g sol}} \right) \times \left(\frac{1.19 \text{ g sol}}{1 \text{ mL sol}} \right) \times \left(\frac{1000 \text{ mL sol}}{1 \text{ L sol}} \right) \times \left(\frac{1 \text{ mol HCl}}{36.46 \text{ g HCl}} \right)$$

■ **Cálculo Numérico:**

$$M = \frac{37.0 \times 1.19 \times 1000}{100 \times 36.46} = \frac{44030}{3646} = \mathbf{12.08 \text{ mol/L} \quad (12.1 \text{ M})}$$

Ejemplo 3: Dilución de Soluciones Concentradas

Enunciado: Se desean preparar 250.0 mL de una solución de ácido sulfúrico (H_2SO_4) a concentración 0.500 M a partir de un ácido concentrado 18.0 M. ¿Qué volumen del ácido concentrado se debe pipetear?

Resolución Paso a Paso:

■ **Ecuación de balance de materia (los moles de soluto permanecen constantes):**

$$n_1 = n_2 \Rightarrow C_1 \cdot V_1 = C_2 \cdot V_2$$

■ **Despeje del volumen inicial requerido (V_1):**

$$V_1 = \frac{C_2 \cdot V_2}{C_1} = \frac{0.500 \text{ M} \times 250.0 \text{ mL}}{18.0 \text{ M}} = \frac{125.0}{18.0} \text{ mL} = \mathbf{6.94 \text{ mL}}$$

- **Instrucción de Seguridad en Laboratorio:** Nunca añadir agua sobre el ácido concentrado. Se deben colocar aproximadamente 150 mL de agua destilada en el matraz aforado, verter lentamente los 6.94 mL de ácido por las paredes, homogeneizar, dejar enfriar a temperatura ambiente y aforar con agua hasta 250.0 mL.

Ejemplo 4: Conversión de ppm a Molaridad en Calidad de Agua

Enunciado: Un análisis de agua potable reporta una concentración de ion plomo (Pb^{2+} , masa atómica 207.2 g/mol) de 0.015 ppm (mg/L). Expresar dicha concentración en Molaridad (mol/L) y en micromolar (μM).

Resolución Paso a Paso:

- **Equivalencia:** $0.015 \text{ ppm} = 0.015 \text{ mg/L} = 0.015 \times 10^{-3} \text{ g/L} = 1.5 \times 10^{-5} \text{ g/L}$.

- **Conversión a Molaridad:**

$$M = \left(\frac{1.5 \times 10^{-5} \text{ g Pb}^{2+}}{1 \text{ L}} \right) \times \left(\frac{1 \text{ mol Pb}^{2+}}{207.2 \text{ g Pb}^{2+}} \right) = 7.24 \times 10^{-8} \text{ mol/L}$$

- **Conversión a micromolar (μM):**

$$C_{\mu\text{M}} = 7.24 \times 10^{-8} \text{ mol/L} \times \left(\frac{10^6 \mu\text{mol}}{1 \text{ mol}} \right) = 0.0724 \mu\text{M}$$

5. Unidades de Concentración en Química

Forma de Concentración	Símbolo	Definición y Expresión Matemática	Unidades Típicas
Molaridad	M	$M = \frac{n_{\text{soluto}} (\text{mol})}{V_{\text{solución}} (\text{L})}$	mol/L, M, $\text{mol} \cdot \text{dm}^{-3}$
Molalidad	m	$m = \frac{n_{\text{soluto}} (\text{mol})}{m_{\text{solvente}} (\text{kg})}$	mol/kg, m
Normalidad	N	$N = \frac{\text{N}^{\circ} \text{ eq-g soluto}}{V_{\text{solución}} (\text{L})} = M \times \theta$	eq/L, N
Fracción Molar	X_i	$X_i = \frac{n_i}{n_{\text{total}}} \quad (\sum X_i = 1)$	Adimensional
Porcentaje en Masa	% m/m	% m/m = $\frac{m_{\text{soluto}}}{m_{\text{solución}}} \times 100$	% (g soluto / 100 g sol)
Porcentaje Masa/Volumen	% m/v	% m/v = $\frac{m_{\text{soluto}} (\text{g})}{V_{\text{solución}} (\text{mL})} \times 100$	% (g soluto / 100 mL sol)
Porcentaje en Volumen	% v/v	% v/v = $\frac{V_{\text{soluto}}}{V_{\text{solución}}} \times 100$	% (mL soluto / 100 mL sol)
Partes por Millón	ppm	$\text{ppm} = \frac{m_{\text{soluto}} (\text{mg})}{m_{\text{solución}} (\text{kg})} \approx \frac{\text{mg}}{\text{L}}$ (en agua diluida)	mg/kg, mg/L, $\mu\text{g/g}$
Partes por Billón	ppb	$\text{ppb} = \frac{m_{\text{soluto}} (\mu\text{g})}{m_{\text{solución}} (\text{kg})} \approx \frac{\mu\text{g}}{\text{L}}$ (en agua diluida)	$\mu\text{g/kg}$, $\mu\text{g/L}$, ng/g

Tabla 1: Expresiones matemáticas y unidades para la concentración de soluciones.

Criterio del Factor Equivalente (θ) en Normalidad

El factor de equivalencia θ depende del tipo de reacción química involucrada:

- **Ácidos:** θ = número de protones H^+ transferibles por molécula (ej. HCl : $\theta = 1$; H_2SO_4 : $\theta = 2$; H_3PO_4 : $\theta = 3$).
- **Bases:** θ = número de grupos OH^- dissociables (ej. NaOH : $\theta = 1$; $\text{Ca}(\text{OH})_2$: $\theta = 2$).
- **Sales:** θ = carga neta total de los cationes o aniones (ej. $\text{Al}_2(\text{SO}_4)_3 \implies \theta = 2 \times (+3) = 6$).
- **Reacciones Redox:** θ = número de electrones ganados o perdidos por fórmula unitaria.

6. Tablas Comparativas de Unidades por Disciplina Científica

A continuación se presentan las tablas organizadas con las magnitudes fundamentales, derivadas y constantes para **Matemáticas, Química, Física y Biología**.

6.1. 1. Tabla de Unidades para Matemáticas e Informática Teórica

Concepto / Magnitud	Símbolo	Unidad	Equivalencia / Descripción Matemática
Ángulo plano (SI)	θ, α	Radián (rad)	$2\pi \text{ rad} = 360^\circ \implies 1 \text{ rad} \approx 57.2958^\circ$
Ángulo sexagesimal	$^\circ, ', ''$	Grado, minuto, segundo	$1^\circ = 60' = 3600'' = \frac{\pi}{180} \text{ rad}$
Ángulo centesimal	grad	Gradián / Gon	$100 \text{ grad} = 90^\circ = \frac{\pi}{2} \text{ rad}$
Ángulo sólido (SI)	Ω	Estereorradián (sr)	Esfera completa = $4\pi \text{ sr}$
Información elemental	b	Bit (bit)	Unidad elemental de entropía informacional (0 ó 1)
Octeto de información	B	Byte (B)	1 Byte = 8 bits
Prefijo binario (IEC)	KiB, MiB	Kibibyte, Mebibyte	1 KiB = $1024 \text{ B} = 2^{10} \text{ B}$; 1 MiB = 2^{20} B
Probabilidad / Frecuencia	P, f_r	Adimensional	Dominio $[0, 1]$ ó escala porcentual $[0\%, 100\%]$
Tasa de variación media	$\Delta y / \Delta x$	Unidades de y/x	Cociente incremental de diferencias finitas
Curvatura de una curva	κ	m^{-1} (inverso de metro)	Magnitud recíproca del radio de curvatura ($1/R$)

Tabla 2: Unidades y estándares de medida en matemáticas aplicadas e informática.

6.2. 2. Tabla de Unidades para Química y Fisicoquímica

Magnitud Química	Símbolo	Unidad Habitual	Equivalencia y Relaciones Clave
Cantidad de sustancia	n	mol (SI), mmol, kmol	$1 \text{ mol} = 6.02214 \times 10^{23} \text{ entidades elementales}$
Masa atómica y molecular	m_a, m_m	uma o Dalton (Da)	$1 \text{ Da} = 1 \text{ uma} = \frac{1}{12} m(^{12}\text{C}) = 1.66054 \times 10^{-27} \text{ kg}$
Masa molar	M	g/mol, kg/mol	$M(\text{H}_2\text{O}) = 18.015 \text{ g/mol}$
Volumen molar gaseoso	V_m	L/mol, m ³ /mol	22.414 L/mol a 0°C y 1 atm
Presión de vapor / gas	P	atm, mmHg, bar, Pa	$1 \text{ atm} = 760 \text{ mmHg} = 101\,325 \text{ Pa} = 1.01325 \text{ bar}$
Entalpía termodinámica	ΔH	kJ/mol, kcal/mol	$1 \text{ kcal} = 4.184 \text{ kJ}$
Entropía molar estándar	S°	J/(mol · K)	Variación entrópica por mol y por Kelvin
Energía libre de Gibbs	ΔG	kJ/mol	Criterio de espontaneidad: $\Delta G = \Delta H - T\Delta S$
Constante universal gases	R	J/(mol · K), $\frac{\text{atm} \cdot \text{L}}{\text{mol} \cdot \text{K}}$	$R = 8.31446 \text{ J/(mol} \cdot \text{K)} = 0.082057 \frac{\text{atm} \cdot \text{L}}{\text{mol} \cdot \text{K}}$
Constante de Faraday	F	C/mol e^-	$F = e \cdot N_A = 96\,485.33 \text{ C/mol de electrones}$
Momento dipolar químico	μ_D	Debye (D), C · m	$1 \text{ D} = 3.33564 \times 10^{-30} \text{ C} \cdot \text{m}$
Potencial de reducción	E°	Voltio (V)	Potencial normal frente al electrodo estándar de H ₂

Tabla 3: Magnitudes fundamentales y constantes en química y termodinámica.

6.3. 3. Tabla de Unidades para Física

6.4. 4. Tabla de Unidades para Biología y Ciencias de la Vida

Magnitud Física	Símb.	Unidad SI	Símb.	Equivalencia en Unidades Base SI
Magnitudes Fundamentales del Sistema Internacional (SI)				
Longitud	l, x, r	Metro	m	Unidad de distancia espacial
Masa	m	Kilogramo	kg	Unidad de inercia y cantidad de materia
Tiempo	t	Segundo	s	Unidad fundamental cronológica
Corriente eléctrica	I	Amperio	A	Flujo de carga: $1 \text{ A} = 1 \text{ C/s}$
Temperatura	T	Kelvin	K	Escala absoluta: $T(\text{K}) = T(^{\circ}\text{C}) + 273.15$
Intensidad luminosa	I_v	Candela	cd	Flujo luminoso por ángulo sólido
Magnitudes Derivadas Principales				
Fuerza / Peso	F, W	Newton	N	$1 \text{ N} = 1 \text{ kg} \cdot \text{m} \cdot \text{s}^{-2}$
Energía / Trabajo / Calor	E, W, Q	Joule (Julio)	J	$1 \text{ J} = 1 \text{ N} \cdot \text{m} = 1 \text{ kg} \cdot \text{m}^2 \cdot \text{s}^{-2}$
Potencia	P	Watt (Vatio)	W	$1 \text{ W} = 1 \text{ J/s} = 1 \text{ kg} \cdot \text{m}^2 \cdot \text{s}^{-3}$
Presión / Esfuerzo	P, σ	Pascal	Pa	$1 \text{ Pa} = 1 \text{ N/m}^2 = 1 \text{ kg} \cdot \text{m}^{-1} \cdot \text{s}^{-2}$
Frecuencia	f, ν	Hertz (Hercio)	Hz	$1 \text{ Hz} = 1 \text{ s}^{-1}$
Carga eléctrica	q, Q	Coulomb (Culombio)	C	$1 \text{ C} = 1 \text{ A} \cdot \text{s}$
Diferencia de Potencial	V	Voltio	V	$1 \text{ V} = 1 \text{ J/C} = 1 \text{ kg} \cdot \text{m}^2 \cdot \text{s}^{-3} \cdot \text{A}^{-1}$
Resistencia eléctrica	R	Ohm (Ω)	Ω	$1 \Omega = 1 \text{ V/A} = 1 \text{ kg} \cdot \text{m}^2 \cdot \text{s}^{-3} \cdot \text{A}^{-2}$
Capacitancia	C	Faradio	F	$1 \text{ F} = 1 \text{ C/V} = 1 \text{ s}^4 \cdot \text{A}^2 \cdot \text{kg}^{-1} \cdot \text{m}^{-2}$
Inductancia magnética	L	Henrio	H	$1 \text{ H} = 1 \text{ Wb/A} = 1 \text{ kg} \cdot \text{m}^2 \cdot \text{s}^{-2} \cdot \text{A}^{-2}$
Inducción magnética	B	Tesla	T	$1 \text{ T} = 1 \text{ N/(A} \cdot \text{m)} = 1 \text{ kg} \cdot \text{s}^{-2} \cdot \text{A}^{-1}$
Flujo magnético	Φ	Weber	Wb	$1 \text{ Wb} = 1 \text{ T} \cdot \text{m}^2 = 1 \text{ kg} \cdot \text{m}^2 \cdot \text{s}^{-2} \cdot \text{A}^{-1}$

Tabla 4: Magnitudes y unidades fundamentales y derivadas de la Física.

Magnitud / Proceso	Símbolo	Unidad Típica	Significado y Contexto Biológico
Dimensión celular/viral	d	μm , nm, Å	Células (μm), orgánulos y virus (nm), enlaces (Å)
Longitud genómica	L_g	pb, kpb, Mpb	Pares de bases de nucleótidos (1 kpb = 1000 pb)
Masa macromolecular	M_r	kDa (Kilodalton)	1 kDa = 1000 Da (proteínas, enzimas, polisacáridos)
Concentración microbiana	N_c	UFC/mL	Unidades Formadoras de Colonias viables por mililitro
Turbidez / Crecimiento	DO ₆₀₀	UA (Adimensional)	Densidad óptica a 600 nm para biomasa en suspensión
Actividad catalítica	U , kat	Unidad (U), Katal (kat)	1 U = 1 μmol soluto transformado/min; 1 kat = 1 mol/s
Tasa específica de crecimiento	μ	h^{-1} , s^{-1}	Cinética de cultivo exponencial: $\mu = \frac{\ln 2}{t_d}$ (t_d : duplicación)
Tasa metabólica basal	TMB	kcal/día, W	Requerimiento energético calórico del organismo en reposo
Dosis de radiación absorbida	D	Gray (Gy)	1 Gy = 1 J/kg de tejido vivo irradiado
Dosis biológica equivalente	H	Sievert (Sv)	Dosis ponderada por efectividad biológica relativa (J/kg)
Coeficiente de sedimentación	S	Svedberg (S)	1 S = 10^{-13} s (ej. Ribosomas procariontes 70S, eucariontes 80S)

Tabla 5: Unidades métricas en biología celular, genética, biotecnología y medicina.

7. Prefijos del Sistema Internacional de Unidades

Múltiplos (Escala Grande)			Submúltiplos (Escala Pequeña)		
Prefijo	Símbolo	Factor Decimal	Prefijo	Símbolo	Factor Decimal
quetta	Q	10^{30}	deci	d	10^{-1}
ronna	R	10^{27}	centi	c	10^{-2}
yotta	Y	10^{24}	mili	m	10^{-3}
zetta	Z	10^{21}	micro	μ	10^{-6}
exa	E	10^{18}	nano	n	10^{-9}
peta	P	10^{15}	pico	p	10^{-12}
tera	T	10^{12}	femto	f	10^{-15}
giga	G	10^9	atto	a	10^{-18}
mega	M	10^6	zepto	z	10^{-21}
kilo	k	10^3	yocto	y	10^{-24}
hecto	h	10^2	ronto	r	10^{-27}
deca	da	10^1	quecto	q	10^{-30}

Tabla 6: Prefijos métricos oficiales del SI actualizados por la CGPM.

8. Consejos Prácticos para Resolución de Problemas

1. **Cuidado con factores elevados a potencias:** Al convertir unidades de superficie (m^2) o volumen (m^3 , cm^3), la equivalencia lineal completa debe elevarse a la potencia respectiva:

$$1 \text{ m} = 100 \text{ cm} \implies (1 \text{ m})^3 = (100 \text{ cm})^3 \implies 1 \text{ m}^3 = 10^6 \text{ cm}^3 = 1000 \text{ L}$$

2. **Equivalencias de volumen frecuentes en laboratorio:**

$$1 \text{ L} = 1 \text{ dm}^3 = 1000 \text{ mL} = 1000 \text{ cm}^3 = 1000 \text{ cc} \implies 1 \text{ mL} = 1 \text{ cm}^3$$

3. **Manejo estricto de cifras significativas:** En cálculos estequiométricos o analíticos, el resultado final debe redondearse al número de cifras significativas del dato con menor precisión. Las constantes exactas por definición (como $1 \text{ min} = 60 \text{ s}$ o números enteros de estequiometría) no restringen la precisión.

Documento técnico compilado como material de estudio y consulta científica.

Definición de Funciones Creadas por el Usuario en Python

Guía Rápida y Ejemplos Prácticos

Resumen Técnico de Programación

20 de agosto de 2026

Resumen

En el lenguaje de programación Python, una función es un bloque de código reutilizable y modular diseñado para ejecutar una tarea específica. Este documento presenta una síntesis estructurada sobre la definición de funciones definidas por el usuario, abarcando desde su sintaxis elemental y manejo de parámetros hasta buenas prácticas como el tipado estático (*type hinting*) y la documentación con *docstrings*.

1. Introducción

La creación de funciones personalizadas (*User-Defined Functions*) permite estructurar el código bajo los principios de la programación modular, reduciendo la redundancia (*DRY - Don't Repeat Yourself*) y facilitando las pruebas unitarias y el mantenimiento del software.

2. Sintaxis Básica y Estructura

Para definir una función en Python se utiliza la palabra reservada **def**, seguida del identificador de la función, paréntesis **()** y dos puntos **::**. El cuerpo de la función debe estar indentado obligatoriamente.

```
1 def saludar():
2     """Imprime un saludo basico."""
3     print("Hola, bienvenido a Python!")
4
5 # Invocacion de la funcion
6 saludar()
```

Listing 1: Estructura básica de una función

3. Parámetros y Argumentos

Las funciones pueden recibir datos de entrada denominados parámetros (en la definición) y argumentos (en la llamada).

3.1. Parámetros Posicionales

Son aquellos que deben coincidir en orden y cantidad con la firma de la función:

```
1 def sumar(a, b):
2     resultado = a + b
3     return resultado
4
```

```
5 # Llamada posicional
6 total = sumar(5, 3) # total = 8
```

Listing 2: Parámetros posicionales

3.2. Parámetros con Valores por Defecto

Permiten asignar valores predeterminados a los parámetros en caso de que el usuario no los proporcione en la llamada:

```
1 def presentar(nombre, rol="Estudiante"):
2     return f"{nombre} tiene el rol de {rol}."
3
4 print(presentar("Ana")) # Usa "Estudiante"
5 print(presentar("Luis", "Docente"))
```

Listing 3: Valores por defecto

3.3. Argumentos por Nombre (*Keyword Arguments*)

Permiten especificar el argumento haciendo referencia explícita al nombre del parámetro, sin importar el orden:

```
1 def dividir(dividendo, divisor):
2     return dividendo / divisor
3
4 res = dividir(divisor=2, dividendo=10) # 5.0
```

Listing 4: Argumentos por clave

4. Retorno de Valores (**return**)

La sentencia **return** finaliza la ejecución de la función y envía un resultado al invocador. Si no se incluye **return**, la función devuelve **None** de forma implícita.

4.1. Retorno Múltiple

Python permite retornar múltiples valores separados por comas, los cuales se empaquetan internamente en una tupla:

```
1 def operaciones(a, b):
2     suma = a + b
3     producto = a * b
4     return suma, producto # Retorna tupla
5
6 s, p = operaciones(4, 5) # Desempaquetado: 9, 20
```

Listing 5: Retorno de múltiples valores

5. Argumentos Arbitrarios: `*args` y `**kwargs`

Cuando no se conoce previamente el número de argumentos que recibirá una función, se recurre a los operadores de empaquetado.

5.1. `*args` (Argumentos Posicionales Variables)

Empaqueta un número indeterminado de argumentos posicionales en una tupla:

```
1 def promedio(*numeros):
2     if not numeros:
3         return 0
4     return sum(numeros) / len(numeros)
5
6 print(promedio(10, 8, 9, 7)) # 8.5
```

Listing 6: Uso de `*args`

5.2. `**kwargs` (Argumentos Nombrados Variables)

Empaqueta argumentos nombrados en un diccionario:

```
1 def info_usuario(**datos):
2     for clave, valor in datos.items():
3         print(f"{clave.capitalize()}: {valor}")
4
5 info_usuario(nombre="Carlos", edad=28, pais="Chile")
```

Listing 7: Uso de `**kwargs`

6. Funciones Anónimas (`lambda`)

Son funciones pequeñas y sin nombre definidas en una sola línea mediante la palabra reservada `lambda`. Son útiles como argumentos en funciones de orden superior como `map()`, `filter()` o `sorted()`:

```
1 # Sintaxis: lambda argumentos: expresion
2 cuadrado = lambda x: x ** 2
3 print(cuadrado(6)) # 36
4
5 # Ordenamiento con funcion lambda
6 puntos = [(1, 2), (4, 1), (5, -1)]
7 puntos.sort(key=lambda p: p[1])
```

Listing 8: Funciones `lambda`

7. Ámbito de las Variables (*Scope*)

Las variables declaradas dentro de una función tienen ámbito local (*local scope*) y no existen fuera de ella. Para modificar una variable global dentro de una función se debe usar la palabra clave `global` (aunque se desaconseja por buenas prácticas).

```
1 x = 10 # Variable global
2
3 def modificar():
4     global x
5     x = 20 # Modifica la variable global
6
7 def local_demo():
8     y = 5 # Variable local
9     return y
```

Listing 9: Alcance de variables

8. Buenas Prácticas y Documentación

8.1. *Docstrings*

Es fundamental documentar el propósito, parámetros y valor de retorno de la función siguiendo las convenciones de la PEP 257:

```
1 def calcular_area_circulo(radio: float) ->
2     float:
3         """Calcula el area de un circulo.
4
5         Args:
6             radio (float): Radio del circulo en
7             metros.
8
9         Returns:
10             float: Area calculada.
11         """
12     import math
13     return math.pi * (radio ** 2)
```

Listing 10: Documentación con `docstring` y `type hints`

8.2. Anotaciones de Tipo (*Type Hints*)

Introducidas en PEP 484, mejoran la legibilidad y permiten la verificación estática del código con herramientas como `mypy`.

9. Tabla de Referencia Rápida

10. Conclusiones

La definición y uso de funciones en Python constituye el pilar básico del diseño de algoritmos estructurados y limpios. Su soporte para parámetros opcionales, desempaquetado flexible (`*args`, `**kwargs`) y funciones de primera clase (*first-class citizens*) convierte a Python en una herramienta versátil y expresiva.

Concepto	Sintaxis / Ejemplo
Definición	<code>def func(): ...</code>
Retorno	<code>return valor</code>
Valor por defecto	<code>def func(x=1): ...</code>
*args (tupla)	<code>def func(*args): ...</code>
kwargs (dict)	<code>def func(kwargs): ...</code>
Lambda	<code>lambda x: x * 2</code>
Anotación tipo	<code>def f(x: int) ->int:</code>

Tabla 1: Resumen de sintaxis de funciones en Python.

La Librería NumPy en Python

Fundamentos, Arquitectura del ndarray y Ejemplos Prácticos

Guía Técnica de Computación Científica

20 de agosto de 2026

Resumen

NumPy (*Numerical Python*) es la librería fundamental para el cálculo numérico y científico en el ecosistema de Python. Proporciona una estructura de datos de alto rendimiento denominada **ndarray** (arreglo multidimensional homogéneo), junto con una vasta colección de funciones matemáticas optimizadas en C/Fortran para álgebra lineal, transformadas de Fourier, estadística y generación de números aleatorios. Este artículo resume los componentes esenciales de NumPy e ilustra su uso mediante ejemplos prácticos.

1. Introducción y Relevancia

El núcleo de la computación científica en Python descansa sobre NumPy. Librerías como **Pandas**, **SciPy**, **Scikit-Learn**, **TensorFlow** y **PyTorch** utilizan o se basan en la arquitectura de arreglos de NumPy.

1.1. ¿Por qué NumPy frente a listas nativas?

A diferencia de las listas estándar de Python, los arreglos de NumPy:

1. **Homogeneidad:** Almacenan elementos del mismo tipo de dato (*data type*), reduciendo la sobrecarga de tipos dinámicos.
2. **Contigüidad en memoria:** Los datos se ubican en bloques continuos de memoria RAM, aprovechando la caché del procesador (*CPU cache locality*).
3. **Vectorización:** Permiten ejecutar operaciones elemento a elemento sin necesidad de bucles **for** explícitos en Python.

2. Importación y el Objeto ndarray

Por convención estándar en la comunidad, NumPy se importa bajo el alias **np**:

```
1 import numpy as np
2
3 # Creación de un arreglo 1D
4 a = np.array([1, 2, 3, 4, 5])
5 print(a)           # [1 2 3 4 5]
6 print(type(a))     # <class 'numpy.ndarray'>
```

Listing 1: Importación y creación básica

2.1. Atributos Principales de un ndarray

Todo arreglo en NumPy cuenta con metadatos descriptivos fundamentales:

- **shape:** Tupla con las dimensiones del arreglo.
- **dtype:** Tipo de datos de los elementos (ej. `int64`, `float32`).
- **ndim:** Número de ejes o dimensiones.
- **size:** Número total de elementos.

```
1 matriz = np.array([[1.0, 2.5, 3.2],
2                   [4.1, 5.0, 6.8]])
3
4 print("Forma (shape):", matriz.shape) # (2, 3)
5 print("Tipo (dtype):", matriz.dtype)  #
6                                     float64
7 print("Dimensiones:", matriz.ndim)    # 2
8 print("Total elementos:", matriz.size)# 6
```

Listing 2: Inspección de atributos

3. Funciones de Creación de Arreglos

NumPy ofrece múltiples constructores especializados para generar arreglos eficientemente:

```
1 # Arreglo de ceros y unos
2 ceros = np.zeros((2, 3))           # Matriz 2x3 de
3                                     0.0
4 unos = np.ones((3, 3), dtype=int)  # Matriz 3
5                                     x3
6
7 # Arreglo con valor constante
8 const = np.full((2, 2), 7)         # [[7, 7], [7,
9                                     7]]
10
11 # Matriz identidad
12 ident = np.eye(3)                  # 3x3 identidad
13
14 # Secuencias numericas
15 rango = np.arange(0, 10, 2)        # [0, 2, 4, 6,
16                                     8]
17 linea = np.linspace(0, 1, 5)       # [0., 0.25,
18                                     0.5, 0.75, 1.]
```

Listing 3: Constructores comunes

4. Generación de Números Aleatorios (numpy.random)

El submódulo `random` permite generar distribuciones estocásticas:

```
1 # Semilla para reproducibilidad
2 np.random.seed(42)
3
4 # Distribucion uniforme en [0, 1)
5 u = np.random.rand(2, 2)
6
7 # Distribucion normal estandar N(0, 1)
8 norm = np.random.randn(1000)
9
10 # Enteros aleatorios en rango [min, max)
11 enteros = np.random.randint(1, 100, size=(4,))
```

Listing 4: Muestreo aleatorio

5. Indexación, Slicing y Filtrado

5.1. Slicing Multidimensional

Se emplea la sintaxis `[inicio:fin:paso]` en cada dimensión separada por comas:

```
1 m = np.array([[10, 20, 30],
2               [40, 50, 60],
3               [70, 80, 90]])
4
5 # Elemento fila 0, columna 2 -> 30
6 elem = m[0, 2]
7
8 # Submatriz: filas 0 a 1, todas las cols
9 sub = m[0:2, :]
10
11 # Todas las filas, ultima columna -> [30, 60, 90]
12 col = m[:, -1]
```

Listing 5: Slicing en matrices

5.2. Indexación Booleana (*Masking*)

Permite filtrar elementos que cumplan una condición lógica de forma directa:

```
1 datos = np.array([15, 22, 8, 45, 19, 31])
2
3 # Mascara booleana
4 mascara = datos > 20 # [F, T, F, T, F, T]
5
6 # Aplicacion de filtro
7 filtrados = datos[mascara] # [22, 45, 31]
```

Listing 6: Filtrado con mascaras booleanas

6. Modificación de Formas (*Reshaping*)

Es posible reconfigurar las dimensiones sin alterar los datos subyacentes:

```
1 v = np.arange(1, 7) # [1, 2, 3, 4, 5, 6]
2
3 # Transformar vector 1D en matriz 2x3
```

```
4 m23 = v.reshape((2, 3))
5
6 # Aplanar matriz a 1D
7 plano = m23.flatten()
8
9 # Transpuesta
10 trans = m23.T # Matriz 3x2
```

Listing 7: Cambio de dimensiones

7. Operaciones Vectorizadas y *Broadcasting*

7.1. Operaciones Elemento a Elemento

Las operaciones aritméticas estándar se ejecutan en paralelo sobre cada posición:

```
1 x = np.array([1, 2, 3])
2 y = np.array([4, 5, 6])
3
4 print(x + y) # [5, 7, 9]
5 print(x * y) # [4, 10, 18]
6 print(x ** 2) # [1, 4, 9]
7 print(np.sqrt(x)) # Raiz cuadrada ufunc
```

Listing 8: Vectorización aritmética

7.2. Regla de *Broadcasting*

El *broadcasting* describe cómo NumPy gestiona arreglos de formas diferentes durante operaciones aritméticas, expandiendo virtualmente la dimensión menor sin duplicar memoria:

```
1 matriz = np.ones((3, 3)) # 3x3 de unos
2 fila = np.array([10, 20, 30]) # 1x3
3
4 # Se suma la fila a cada una de las 3 filas
5 resultado = matriz + fila
6 # [[11, 21, 31],
7 #   [11, 21, 31],
8 #   [11, 21, 31]]
```

Listing 9: Ejemplo de broadcasting

8. Estadística y Reducción por Ejes

NumPy incluye funciones de reducción estadística optimizadas:

```
1 datos = np.array([[1, 2, 3],
2                   [4, 5, 6]])
3
4 print("Suma total:", np.sum(datos)) # 21
5 print("Promedio:", np.mean(datos)) # 3.5
6 print("Desv. Estandar:", np.std(datos)) # 1.7078
7 print("Maximo:", np.max(datos)) # 6
8
9 # Operaciones por eje:
10 # axis=0 -> a lo largo de las columnas
11 print("Suma columnas:", np.sum(datos, axis=0)) # [5, 7, 9]
12 # axis=1 -> a lo largo de las filas
```

```

13 print("Suma filas:", np.sum(datos, axis=1))
    # [6, 15]

```

Listing 10: Estadística descriptiva y ejes

9. Álgebra Lineal (numpy.linalg)

El módulo `linalg` implementa rutinas avanzadas de álgebra matricial:

```

1 A = np.array([[2, 1],
2               [1, 3]])
3 b = np.array([8, 13])
4
5 # Multiplicación matricial (operador @)
6 B = np.array([[1, 0], [0, 2]])
7 prod = A @ B
8
9 # Determinante e inversa
10 det_A = np.linalg.det(A)
11 inv_A = np.linalg.inv(A)
12
13 # Solución de sistema lineal A * x = b
14 x = np.linalg.solve(A, b) # x = [2.2, 3.6]
15
16 # Autovalores y autovectores
17 valores, vectores = np.linalg.eig(A)

```

Listing 11: Operaciones matriciales

10. Tabla Resumen de Métodos Clave

Función / Método	Propósito
<code>np.array(lista)</code>	Crear un ndarray
<code>np.zeros/ones(shape)</code>	Inicializar con 0s o 1s
<code>np.arange(start, stop, step)</code>	Rango con paso fijo
<code>np.linspace(a, b, num)</code>	Intervalo equi-espaciado
<code>arr.reshape(new_shape)</code>	Reorganizar dimensiones
<code>arr[condicion]</code>	Filtrado booleano
<code>np.dot(A, B) / A @ B</code>	Producto matricial
<code>np.sum(arr, axis)</code>	Agregación por eje
<code>np.linalg.solve(A, b)</code>	Resolver sistemas $Ax = b$

Tabla 1: Funciones esenciales de NumPy.

11. Conclusión

NumPy constituye la infraestructura computacional sobre la que se erige la ciencia de datos, el aprendizaje automático y la física computacional moderna en Python. Su combinación de alto rendimiento mediante código compilado en C y sintaxis concisa y expresiva en Python lo convierte en una herramienta indispensable.